



The Design Loop: Free Guide

🗣️ Claude Design has taste. It just can't judge its own work.

The context that builds a piece is the context that grades it — a chef reviewing their own restaurant. Five stars, every time. The Design Loop fixes exactly that: it moves the judging into **fresh contexts** that never saw the work being made.

Parallel agents buy you speed. *Fresh context buys you the result.*

Source of the Gauntlet loop

This method is an adaptation of the **Gauntlet Loop**, invented by **Matt Shumer**. He named it, defined it, and built the original — a Three.js FPS from a single prompt that did 3.8M views. Everything on this page is a variation on his idea.



Matt Shumer introduces the Gauntlet Loop

- His founding post — "I'm officially calling this the Gauntlet Loop"
- How to run one, and why it works for more than games
- The original prompt, verbatim (GitHub)
- The method write-up

Start here

1. Export your system

In **Claude Design**, ask it to write out your design system as a single `design-system.md`. Do this **once** per project.

2. Install the skill

Drop `SKILL.md` into `.claude/skills/design-loop/` in your project. Claude Code reads it automatically.

3. Run it

Type `/design-loop` and answer three questions. That's it — you type twice in the whole run.

⚠ **Reserve this for your hero asset.** Every round is a build plus three judgments. It is genuinely token-hungry. Use it on the landing page or the flagship carousel — not on the whole site. Your weekly limit is the real hard stop, and **you are the brake**.

The skill

The whole thing is one file. Save it as `.claude/skills/design-loop/SKILL.md` inside your project, then type `/design-loop`.

```
---
name: design-loop
description: Takes a goal and a real-world reference, extracts what actually makes the r
---

# Design Loop

Four phases: interview, preflight, teardown, loop. Do not skip ahead. Do not start build:

## Phase 1: Interview

Ask exactly these three, together, then stop and wait.

1. What are you building, and how long or how big?
2. Name something that already does this brilliantly. A site, a video, a doc, anything I
3. Any files I should work from? Design system, brand doc, script, existing draft.

If they name something vague ("Apple's website", "good SaaS design"), push once for the
If they say skip on question 2, propose three candidate bars, one line each on why, and

## Phase 2: Preflight

A check, not a question. Run it before any work and report in one block.

- Fetch the bar now. Screenshot the URL or read the file. If it is blocked or missing, s
- Confirm you can render our output: screenshots for a site, a filmstrip of frames for a
- Name any generation tools the goal needs (image, video, voice) and confirm they are co
- Confirm the input files exist: design-system.md, brand doc, script.

Then print: what is working, what is missing, and **which critic goes blind** if somethin

## Phase 3: Teardown

Read the reference properly and write 5 to 7 mechanisms to `bar.md`.

Mechanisms, not adjectives. "Feels premium" is useless. These are useful:
- headline is 5x body size, three type sizes total
- one accent colour, used at most twice per screen
- motion always resolves in one direction
- nothing animates for under 400ms
- whitespace above the fold is at least 40% of the frame

Every line must be something a critic can check by looking. Show `bar.md` to the user be

## Phase 4: Loop

Split the goal into the smallest pieces that can be improved and judged on their own. Yo

For each piece: fan out a builder, then three critics, each with fresh context and no kn

- **Brief critic** judges against the stated goal only. Does it do the thing? Ignore aes
- **System critic** judges against `design-system.md` only. Objective adherence.
- **Craft critic** judges against `bar.md` and rendered output only. Put ours next to th

Write each critic's brief yourself, adapted to this specific goal. Do not reuse generic

Rules:
- Critics are harsh. Praise is not useful.
- Critics judge rendered output, never the code. Reading the implementation makes a crit
- Binary verdicts, not scores. Scores drift upward every round.
- All three must pass. Any fail goes back to the builder with the single biggest gap nam
- No fixed round count. The exit is winning, or the user stopping the run.

Keep a live progress page updating as work evolves: piece status, each critic's verdict,

## Cost

There is no reliable self-reported token cost, so do not pretend to show one. Show round

If the user names a ceiling, treat it as a checkpoint: pause and ask before continuing p

## What breaks this

- A vague bar. By far the most common failure.
- The builder judging its own work. Critics need fresh context.
- A soft critic. Binary job, not a score.
- A fixed round count. The exit is winning.
- Over-specifying. Every extra instruction is one fewer decision the model makes with it:

< >
```

The three critics

You never write a critic. The three roles are hardcoded so they never converge into the same opinion — but each one's specific brief is written per run, because "does it hit the brief" means something different for an animation than for a pricing page.

Critic	Judges against	Model	Why
Brief	The stated goal only, ignoring aesthetics	Sonnet	Simple judgment, no vision needed
System	<code>design-system.md</code> only	Haiku	Mechanical adherence checking
Craft	<code>bar.md</code> and rendered frames, never the code	Strongest available	Never downgrade this one. A cheap craft critic approves everything and the loop dies on round one.

What makes a bar actually work

The teardown converts your reference into **mechanisms a critic can check by looking**. This is the step that does the real work, and it is also the step people get wrong. Adjectives are unfalsifiable; measurements are not.

✗ Useless

- Feels premium
- Clean and modern
- Good use of whitespace
- Strong visual hierarchy

✓ Checkable

- Headline is 5x body size
- Three type sizes total, no more
- One accent colour, max twice per screen
- Nothing animates under 400ms

A vague bar is the single most common reason this fails. The critic invents its own comparison and approves everything on round one.

Cost, honestly

The cost line inside any prompt is a **wish, not a brake** — the model can't reliably see its own spend. Here's a real, publicly reported number from someone running this method to build an F1 game from a single prompt:

🕒 19 hrs

one run

💰 1.7B

tokens

👥 137

agents

💵 ~\$1,200

API cost

That is the extreme end — a 19-hour game build, not a landing page. But it's the honest shape of the trade: **this is powerful because it's expensive**. Real controls, in order of what actually works:

1. Watch it and stop it. You are the brake.
2. Cap the **pieces** to three or four, not the rounds. Every extra piece multiplies the run.
3. Let your weekly limit be the hard stop.

What this version adds

Addition	Why it matters
A teardown pass into <code>bar.md</code>	Converts a reference into checkable mechanisms <i>before</i> any building starts
Three critics with distinct briefs	Each can fail work the other two would pass — one "harsh critic" collapses into a single opinion
A preflight check	Verifies the critic can actually see before you waste a whole run
Model tiering across critics	Cheap where judgment is mechanical, strongest where it's taste

🧠 The one thing to remember

A critic that shares memory with the builder is grading its own homework. Everything else here is detail.